Table of Contents

1. Introduction
2. Architectural Views
    2.1. VIEW-CONTEXT
    2.2. VIEW-CONTAINER
    2.3. VIEW-DEPLOYMENT
3. Design Chapters
    3.1. Bank Link Fallback Behavior
        3.1.1. Chapter Scope Diagram
        3.1.2. Reference Map
    3.2. Decline Handling And Support Flow
        3.2.1. Chapter Scope Diagram
        3.2.2. Reference Map
    3.3. Fraud Check Integration
        3.3.1. Chapter Scope Diagram
        3.3.2. Reference Map
    3.4. Payment Authorization Flow
        3.4.1. Chapter Scope Diagram
        3.4.2. Reference Map
    3.5. High-Risk Review Escalation
        3.5.1. Chapter Scope Diagram
        3.5.2. Reference Map
4. Reference Index
    4.1. Actor(s)
        4.1.1. Customer (ACT-CUSTOMER)
        4.1.2. Support agent (ACT-SUPPORT)
    4.2. System(s)
        4.2.1. Bank gateway (EXT-BANK-GW)
        4.2.2. Fraud detection system (SYS-FRAUD)
        4.2.3. Payments system (SYS-PAYMENTS)
    4.3. Container(s)
        4.3.1. Checkout API (CONT-CHECKOUT-API)
        4.3.2. Payment Engine (CONT-PAYMENT-ENGINE)
        4.3.3. Risk Scorer (CONT-RISK-SCORER)
    4.4. Component(s)
        4.4.1. Authorization Orchestrator (COMP-AUTH-ORCHESTRATOR)
        4.4.2. Rule Evaluator (COMP-RULE-EVALUATOR)
    4.5. Event(s)
        4.5.1. Bank link is unavailable (EVT-BANK-LINK-UNAVAILABLE)
        4.5.2. Payment authorization is requested (EVT-PAYMENT-AUTH-REQUESTED)
        4.5.3. Payment is declined (EVT-PAYMENT-DECLINED)
        4.5.4. Risk score is high (EVT-RISK-HIGH)
        4.5.5. Support agent is notified (EVT-SUPPORT-NOTIFIED)
    4.6. State(s)

4.6.1. Bank link is available (STATE-BANK-LINK-UP)
4.6.2. Checkout is active (STATE-CHECKOUT-ACTIVE)
4.6.3. Payment review is pending (STATE-REVIEW-PENDING)
4.7. Feature(s)
4.7.1. Fraud check is enabled (FEAT-FRAUD-CHECK)
4.8. Mode(s)
4.8.1. Normal mode is active (MODE-NORMAL)
4.9. Condition(s)
4.9.1. 3DS verification is required (COND-3DS-REQUIRED)
4.9.2. Risk threshold is exceeded (COND-RISK-THRESHOLD-EXCEEDED)
4.10. Data term(s)
4.10.1. Decline reason (DATA-DECLINE-REASON)
4.10.2. Payment id (DATA-PAYMENT-ID)
4.10.3. Risk score (DATA-RISK-SCORE)
5. Requirements Appendix
5.1. REQ-PAY-001
5.2. REQ-PAY-002
5.3. REQ-PAY-003
5.4. REQ-PAY-004
5.5. REQ-PAY-005
5.6. REQ-PAY-006

---

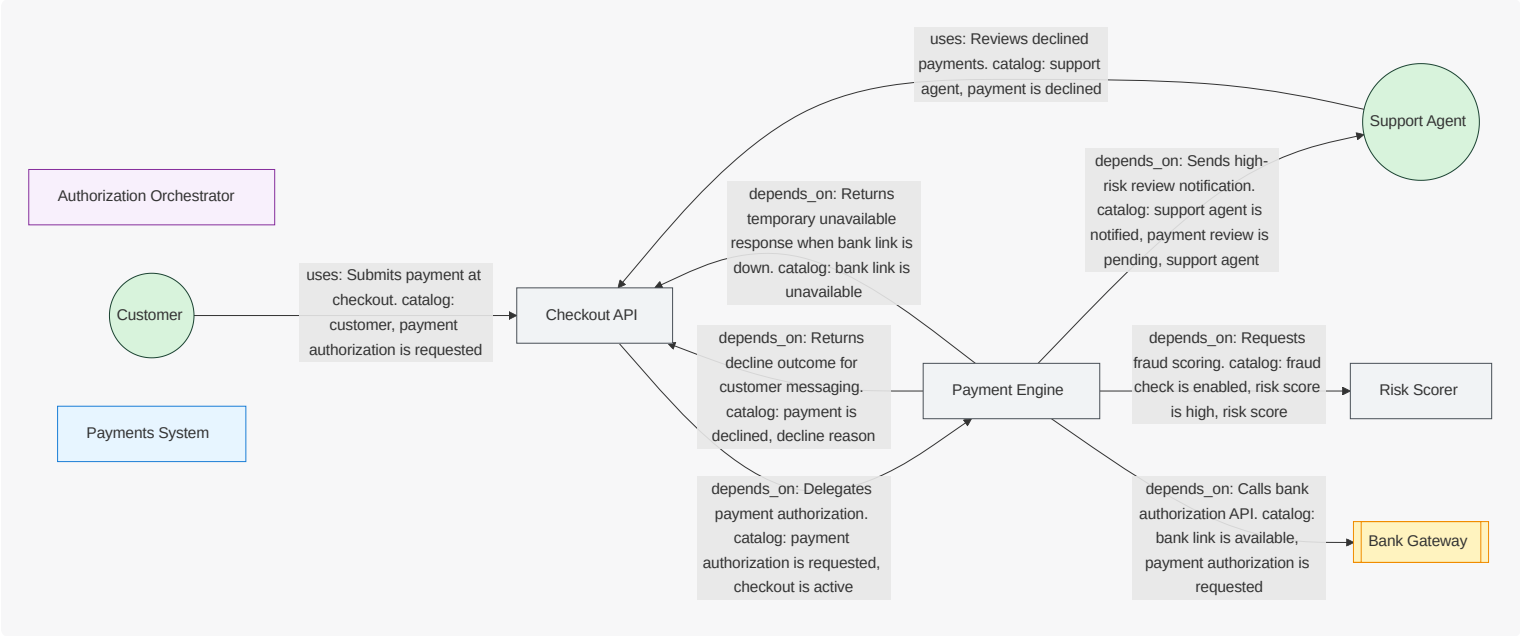
1. Introduction

This architecture description is generated from `architecture.yml`, `requirements.yml`, and `design.yml`. Each design chapter is anchored on catalog references, while architecture elements and requirement links are derived automatically.

2. Architectural Views

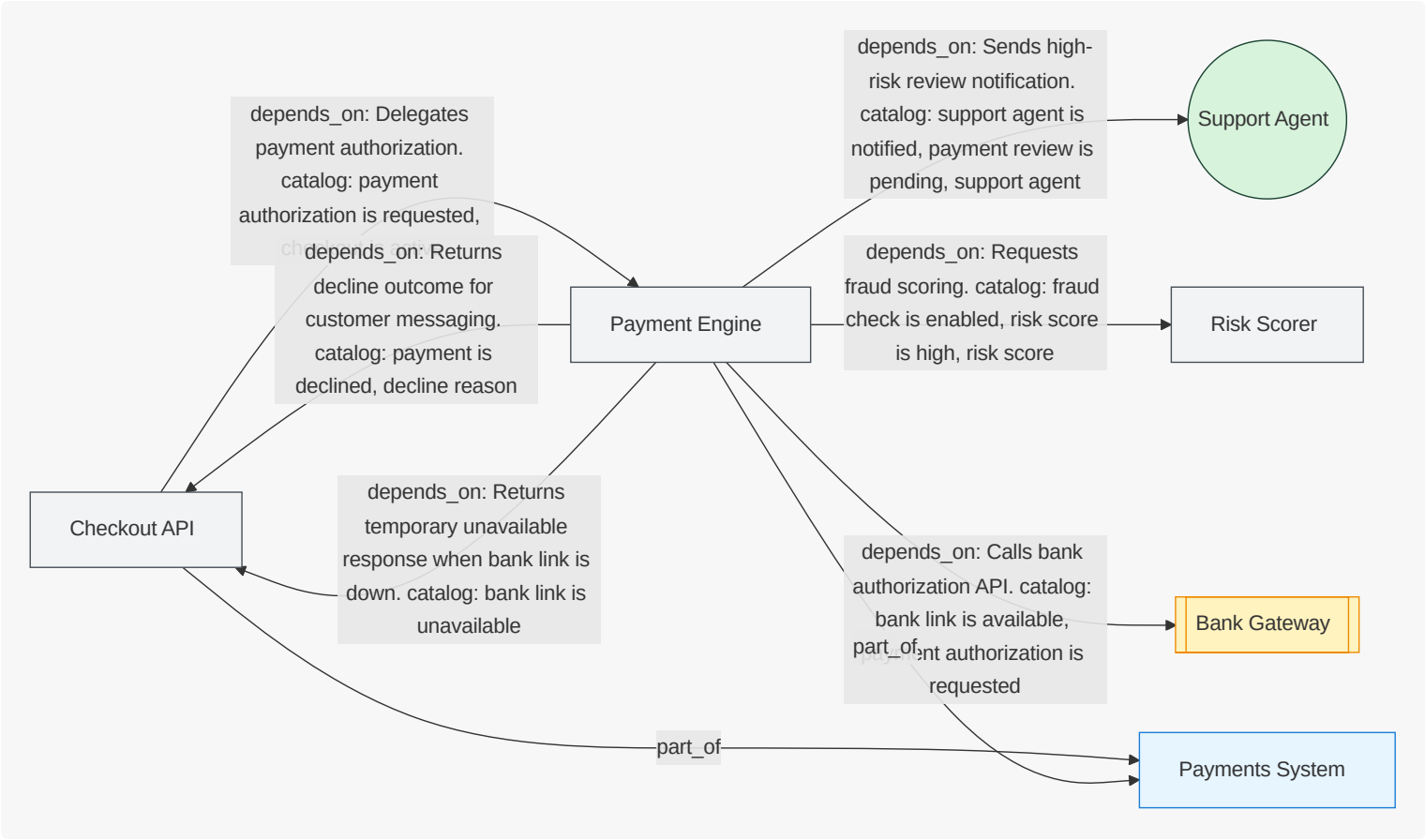
2.1. VIEW-CONTEXT

The following view is the full model projection for `VIEW-CONTEXT`.



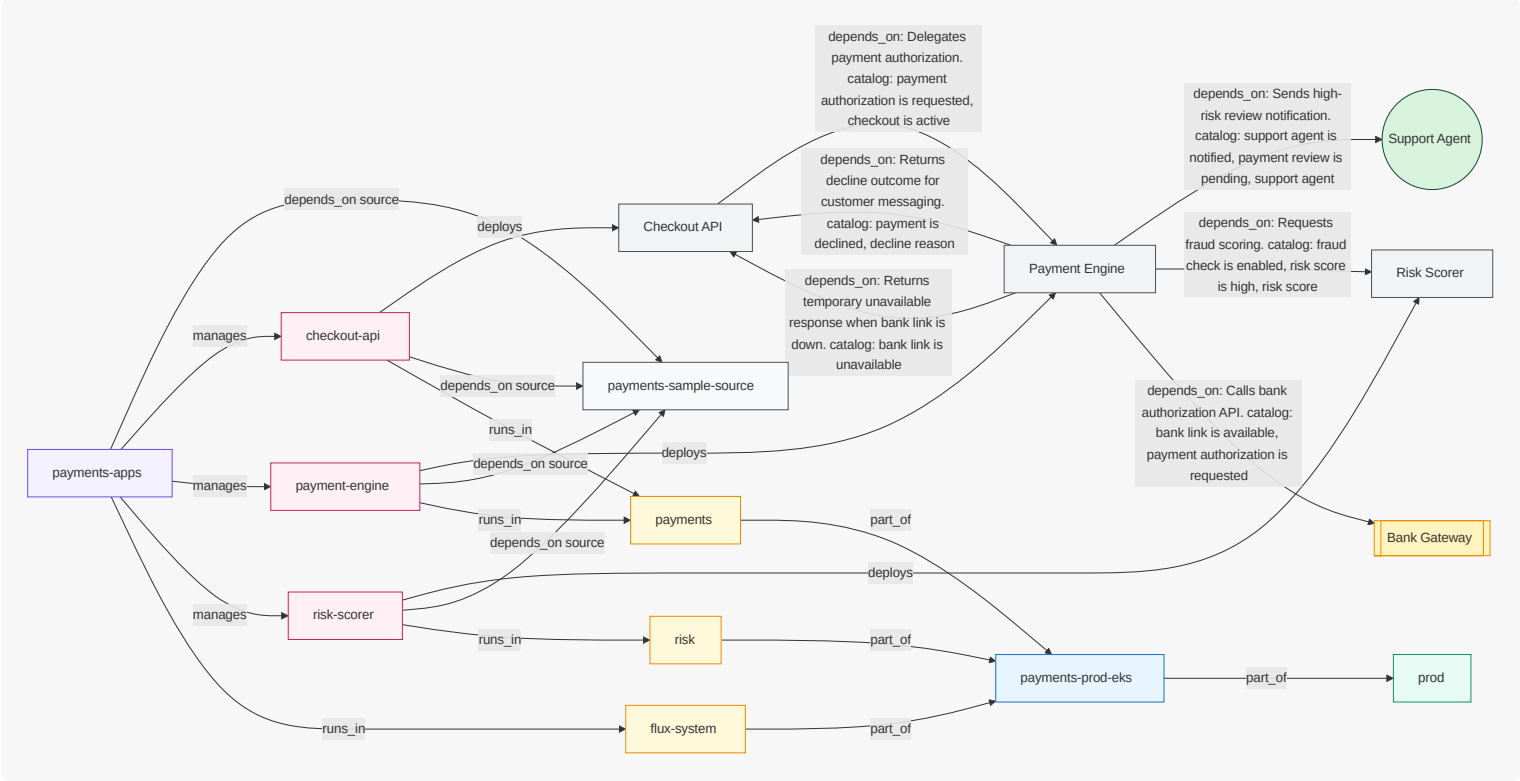
2.2. VIEW-CONTAINER

The following view is the full model projection for `VIEW-CONTAINER`.



2.3. VIEW-DEPLOYMENT

The following view is the full model projection for `VIEW-DEPLOYMENT`.



3. Design Chapters

3.1. Bank Link Fallback Behavior

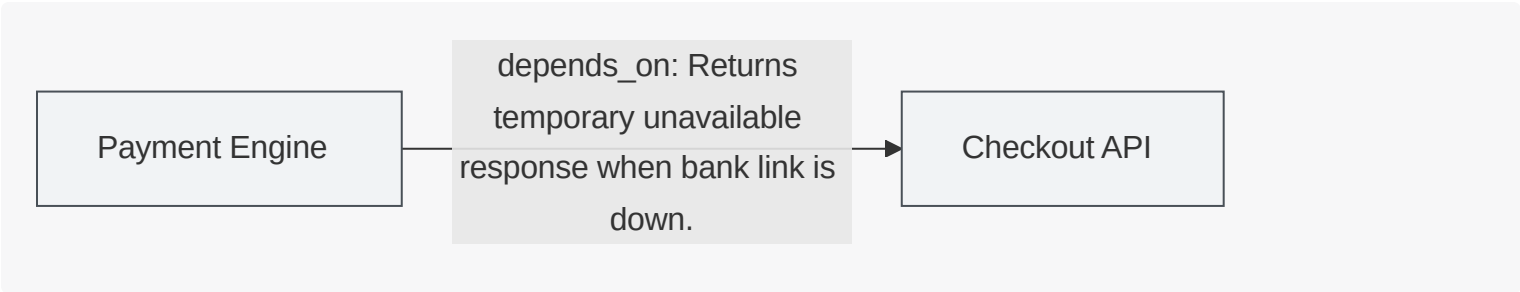
Fallback behavior is activated when bank link is unavailable occurs during authorization. In this condition Payment Engine must stop external authorization attempts and return a deterministic temporary unavailable response to Checkout API.

Checkout API converts that response into user guidance that distinguishes transient infrastructure failure from card or account decline. This prevents false negative messaging and reduces support agent load caused by misleading decline prompts.

This complements, but does not replace, the normal bank-link-up authorization path.

3.1.1. Chapter Scope Diagram

The following diagram shows only direct dependencies tagged by this chapter's catalog references.



3.1.2. Reference Map

- Derived Architecture Refs: Checkout API, Payment Engine
- Derived Requirement Refs: REQ-PAY-006

- Direct Relationships: Payment Engine → Checkout API (depends_on: Returns temporary unavailable response when bank link is down.)

3.2. Decline Handling And Support Flow

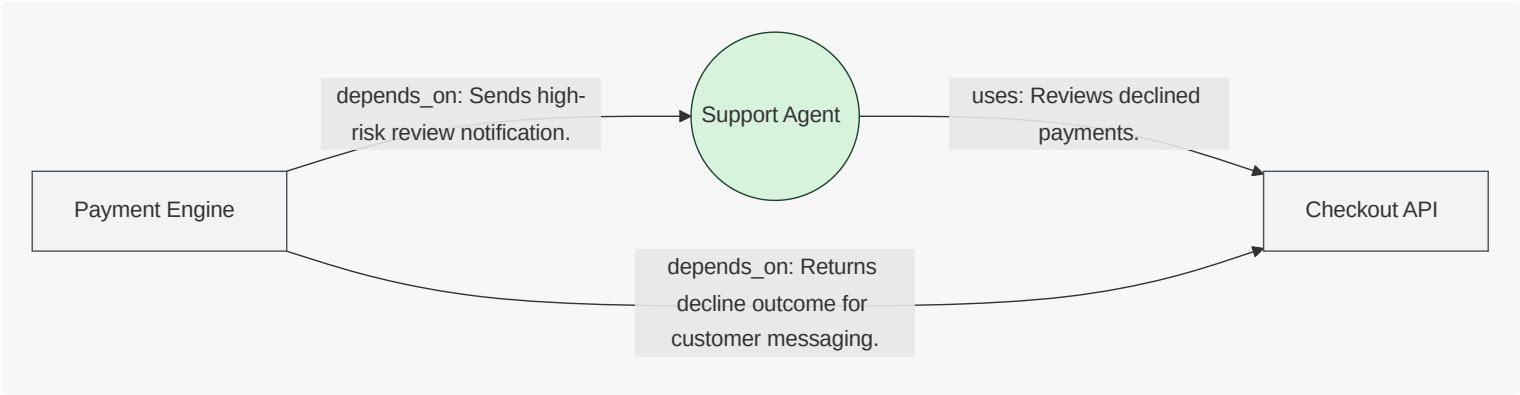
When payment is declined is produced, Payment Engine returns a decline payload to Checkout API containing customer-safe messaging and decline reason details. Checkout API then renders clear feedback and retry guidance so users can recover without ambiguous failure states.

support agent is linked to the same decline context to allow post-event triage, root-cause review, and operational communication. This keeps user-facing behavior and support agent-facing diagnosis aligned to the same canonical event and data terms.

The chapter covers both customer messaging and support agent-side operational follow-up.

3.2.1. Chapter Scope Diagram

The following diagram shows only direct dependencies tagged by this chapter's catalog references.



3.2.2. Reference Map

- Derived Architecture Refs: support agent, Checkout API, Payment Engine
- Derived Requirement Refs: REQ-PAY-003, REQ-PAY-004
- Direct Relationships: support agent → Checkout API (uses: Reviews declined payments.); Payment Engine → support agent (depends_on: Sends high-risk review notification.); Payment Engine → Checkout API (depends_on: Returns decline outcome for customer messaging.)

3.3. Fraud Check Integration

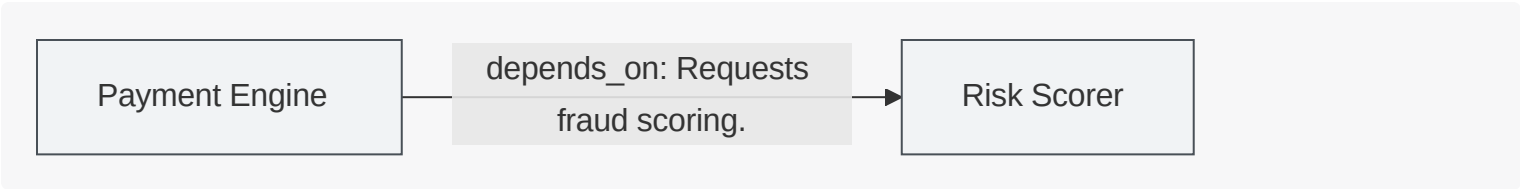
Fraud evaluation is a mandatory branch for transactions where fraud check is enabled is enabled. Payment Engine calls Risk Scorer before final authorization, and the risk pipeline computes risk score from request metadata, amount, and behavioral signals.

If the resulting score indicates risk score is high, the authorization path must transition away from immediate approval and into controlled review handling. If the score remains below threshold, the payment can continue through the normal bank authorization branch.

This chapter defines the boundary between fast-path processing and human-review escalation.

3.3.1. Chapter Scope Diagram

The following diagram shows only direct dependencies tagged by this chapter's catalog references.



3.3.2. Reference Map

- Derived Architecture Refs: Payment Engine, Risk Scorer
- Derived Requirement Refs: REQ-PAY-002, REQ-PAY-004, REQ-PAY-005
- Direct Relationships: Payment Engine → Risk Scorer (depends_on: Requests fraud scoring.)

3.4. Payment Authorization Flow

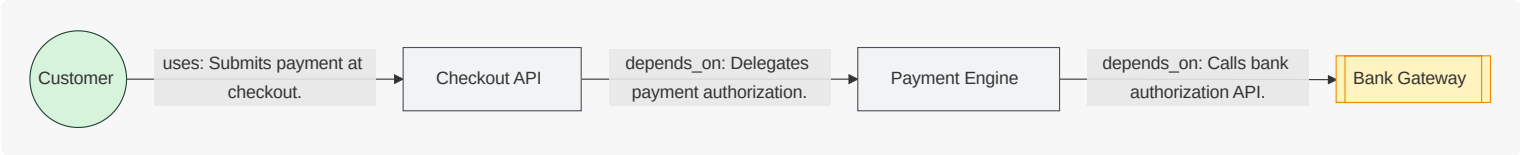
The payment authorization flow starts when a customer submits checkout data and the system raises payment authorization is requested while checkout is active is true. Checkout API creates a payment session key and forwards the authorization request to Payment Engine.

Payment Engine coordinates internal checks and sends the external authorization call to bank gateway only when bank link is available is satisfied. The payment id term is carried end-to-end so every service invocation and auditable decision can be correlated across logs, traces, and support agent tooling.

This is the nominal happy-path before decline or review branches are considered.

3.4.1. Chapter Scope Diagram

The following diagram shows only direct dependencies tagged by this chapter's catalog references.



3.4.2. Reference Map

- Derived Architecture Refs: customer, Checkout API, Payment Engine, bank gateway
- Derived Requirement Refs: REQ-PAY-001, REQ-PAY-002, REQ-PAY-004, REQ-PAY-005
- Direct Relationships: customer → Checkout API (uses: Submits payment at checkout.); Checkout API → Payment Engine (depends_on: Delegates payment authorization.); Payment Engine → bank gateway (depends_on: Calls bank authorization API.)

3.5. High-Risk Review Escalation

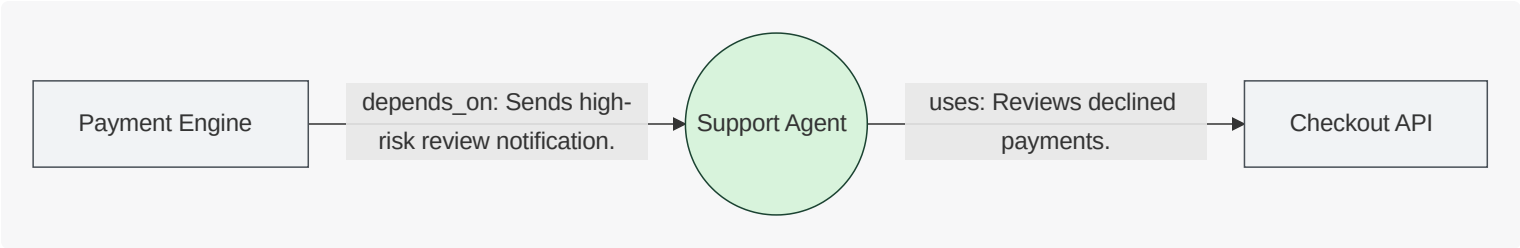
Review escalation applies when risk logic determines the transaction cannot be auto-approved. Payment Engine places the transaction into payment review is pending and emits support agent is notified so manual intervention is guaranteed and auditable.

support agent receives enough context to decide whether to release, reject, or request additional verification. This chapter intentionally describes a controlled pause state instead of a hard failure so business workflows can continue safely under supervision.

Checkout API provides status visibility while support agent decisions are made.

3.5.1. Chapter Scope Diagram

The following diagram shows only direct dependencies tagged by this chapter's catalog references.



3.5.2. Reference Map

- Derived Architecture Refs: support agent, Checkout API, Payment Engine
- Derived Requirement Refs: REQ-PAY-004
- Direct Relationships: support agent → Checkout API (uses: Reviews declined payments.); Payment Engine → support agent (depends_on: Sends high-risk review notification.)

4. Reference Index

This section combines implementation elements and glossary terms in one place.

4.1. Actor(s)

4.1.1. Customer (ACT-CUSTOMER)

End user submitting payment details during checkout.

Aliases: `buyer`

4.1.2. Support agent (ACT-SUPPORT)

Operations role that investigates declines and high-risk review cases.

Aliases: `support`

4.2. System(s)

4.2.1. Bank gateway (EXT-BANK-GW)

External banking endpoint used for card authorization and decline decisions.

Aliases: `bank API`

4.2.2. Fraud detection system (SYS-FRAUD)

Service that evaluates transaction risk and classifies payments for approval or review.

Aliases: `fraud service`

4.2.3. Payments system (SYS-PAYMENTS)

Core platform that orchestrates checkout, authorization, and customer payment outcomes.

Aliases: `payment platform`

4.3. Container(s)

4.3.1. Checkout API (CONT-CHECKOUT-API)

Checkout API runs as a container-level runtime unit using Go.

Aliases: *none*

Parent: payments system

4.3.2. Payment Engine (CONT-PAYMENT-ENGINE)

Payment Engine runs as a container-level runtime unit using Rust.

Aliases: *none*

Parent: payments system

4.3.3. Risk Scorer (CONT-RISK-SCORER)

Risk Scorer runs as a container-level runtime unit using TypeScript.

Aliases: *none*

Parent: fraud detection system

4.4. Component(s)

4.4.1. Authorization Orchestrator (COMP-AUTH-ORCHESTRATOR)

Authorization Orchestrator is a component-level responsibility within its parent container.

Aliases: *none*

Parent: Payment Engine

4.4.2. Rule Evaluator (COMP-RULE-EVALUATOR)

Rule Evaluator is a component-level responsibility within its parent container.

Aliases: *none*

Parent: Risk Scorer

4.5. Event(s)

4.5.1. Bank link is unavailable (EVT-BANK-LINK-UNAVAILABLE)

Event raised when payment engine cannot reach the external bank gateway.

Aliases: `bank link down`

4.5.2. Payment authorization is requested (EVT-PAYMENT-AUTH-REQUESTED)

Event raised when checkout asks payment engine to authorize a transaction.

Aliases: `payment auth requested`

4.5.3. Payment is declined (EVT-PAYMENT-DECLINED)

Event indicating authorization was rejected and customer decline handling must execute.

Aliases: `payment declined`

4.5.4. Risk score is high (EVT-RISK-HIGH)

Event emitted when computed fraud risk exceeds the configured high-risk threshold.

Aliases: `high risk`

4.5.5. Support agent is notified (EVT-SUPPORT-NOTIFIED)

Event indicating manual review notification was sent to support operations.

Aliases: `support notified`

4.6. State(s)

4.6.1. Bank link is available (STATE-BANK-LINK-UP)

Connectivity state showing bank gateway communication is currently healthy.

Aliases: `bank link up`

4.6.2. Checkout is active (STATE-CHECKOUT-ACTIVE)

Processing state where the user is actively submitting or confirming payment.

Aliases: `checkout active`

4.6.3. Payment review is pending (STATE-REVIEW-PENDING)

Transaction state requiring manual support review before final disposition.

Aliases: `review pending`

4.7. Feature(s)

4.7.1. Fraud check is enabled (FEAT-FRAUD-CHECK)

Feature flag controlling whether risk scoring is required before authorization.

Aliases: `fraud check`

4.8. Mode(s)

4.8.1. Normal mode is active (MODE-NORMAL)

Standard operating mode with full external dependency usage enabled.

Aliases: `normal mode`

4.9. Condition(s)

4.9.1. 3DS verification is required (COND-3DS-REQUIRED)

Condition indicating strong customer authentication must be performed.

Aliases: `3DS required`

4.9.2. Risk threshold is exceeded (COND-RISK-THRESHOLD-EXCEEDED)

Condition indicating risk score crossed a policy threshold for escalation.

Aliases: `risk above threshold`

4.10. Data term(s)

4.10.1. Decline reason (DATA-DECLINE-REASON)

Structured reason code or message explaining why authorization failed.

Aliases: `decline code`

4.10.2. Payment id (DATA-PAYMENT-ID)

Stable identifier used to correlate a payment across services and records.

Aliases: *none*

4.10.3. Risk score (DATA-RISK-SCORE)

Numeric fraud score produced by risk evaluation logic.

Aliases: *none*

5. Requirements Appendix

5.1. REQ-PAY-001

While checkout is active, when payment authorization is requested, the payments system shall create a payment session and request authorization from bank gateway.

Notes: Core authorization path from checkout to bank gateway.

5.2. REQ-PAY-002

Where fraud check is enabled, when payment authorization is requested, the fraud detection system shall calculate a risk score before approval.

Notes: Fraud scoring gate before approval.

5.3. REQ-PAY-003

If payment is declined, then the payments system shall present a decline reason and a retry option to customer.

Notes: Decline handling and customer feedback.

5.4. REQ-PAY-004

While checkout is active, when payment authorization is requested and risk score is high, the payments system shall place the payment in payment review is pending state and notify support agent.

Notes: Escalation flow for high-risk payments.

5.5. REQ-PAY-005

Where fraud check is enabled, when payment authorization is requested, the payments system shall persist an audit record with payment id and risk score.

Notes: Auditability for authorization decisions.

5.6. REQ-PAY-006

If bank link is unavailable, then the payments system shall present a temporary unavailable message to customer.

Notes: Fallback behavior when external banking path is down.